3. URL Shortener (TinyURL / bit.ly)

> **Interviewer:** Design a URL shortener — something like TinyURL or bit.ly. A user submits a long URL, we hand back a short one; anyone who visits the short link gets redirected to the original. Simple enough on paper — walk me through it.

Candidate: It sounds simple, and the write side genuinely is. The whole problem is that this system is **almost pure reads at extreme scale** — every redirect is on some stranger's critical path, and the ratio of clicks to link-creations can be 100:1 or higher. Let me clarify before I design.

---

1. Clarifying the requirements

> **Candidate:** A few questions: 1. Do we need **custom aliases** (`bit.ly/my-brand`), or is auto-generated enough? 2. Do links **expire**, or live forever? 3. Do we need **click analytics** (count, referrer, geo), and can that be approximate/delayed? 4. Is this a **global** product — users and links all over the world — or single-region? 5. 301 (permanent) or 302 (temporary) redirect — does the interviewer have a preference, or is that mine to justify?
>
> **Interviewer:** Yes to custom aliases. Links can optionally expire. Yes, we want click counts — approximate and a bit delayed is totally fine. Assume it's global, used from everywhere. And yes, justify the redirect status code yourself.

Candidate: Good, that pins down the shape. Let me write requirements.

Functional - `POST /shorten` → `{ longUrl, customAlias?, expiresAt? }` → `{ shortUrl }`. - `GET /{code}` → HTTP redirect to the original long URL. - Optional custom alias (user picks the code); otherwise we generate one. - Optional expiration; expired codes 404 (or a "this link has expired" page). - Click analytics: count clicks, maybe referrer/geo — asynchronous, eventually consistent.

Non-functional — this is where the problem actually lives - **Read:write ≈ 100:1** — this is fundamentally a read-scaling problem, not a write one. - **Redirect latency** must be tiny (sub-50ms) — it sits directly in a real person's page load. - **High availability on the read path** — a shortener that's down breaks every link that points at it, everywhere on the internet, right now. - **Uniqueness** of generated codes at high write throughput, without a global lock. - **Global** users → multi-region reads with low latency everywhere.

---

2. Estimation — sizing the read/write split

Candidate: Let me size both sides, because the *gap* between them drives the architecture.

```
Assume 100M new short links created per month (writes):
  writes/sec ≈ 100,000,000 / (30 * 86,400) ≈ 40 writes/sec        <- trivial

Assume a 100:1 read:write ratio (clicks vs. creations):
  reads/sec  ≈ 40 * 100 = 4,000 reads/sec average
  peak (viral links, diurnal spikes) ≈ 10x average ≈ 40,000 reads/sec

Storage, 5-year retention:
  100M/month * 12 * 5 = 6B records
  ~500 bytes/record (code + long URL + metadata) => ~3 TB total  (small)

Short code space, base62 [a-zA-Z0-9]:
  62^6  ≈ 56.8 billion combinations
  62^7  ≈ 3.5 trillion combinations
  => 7 chars comfortably covers 6B+ links for decades; I'll use 7.

=> 40 writes/sec is nothing. 40,000 reads/sec (bursting far higher on a
   viral link) is the whole game. Everything downstream — cache, CDN,
   replica count, region layout — is sized for READS, not writes.
```

Candidate: That 100:1 skew is the headline number. If I get the read path wrong, the product is broken; if I get the write path merely *adequate*, nobody notices.

---

3. V1 — the simplest thing that could work

Candidate: Let me draw the naive version first, single region, no cache, so we have a baseline to improve on.

```
  [ Client ] --POST /shorten--> [ App Server ] --> [ Database ]
                                       |              INSERT (code, long_url)
                                       |
  [ Client ] --GET /{code}-----> [ App Server ] --> [ Database ]
                                       |              SELECT long_url WHERE code=?
                                       ▼
                                302 Location: <longUrl>
```

Why it's not good enough: every single redirect — all 40,000/sec at peak, skewed brutally toward a handful of viral links (power law: a tiny fraction of codes get most clicks) — hits the primary database directly. That database now has to serve both the write path *and* an enormous, spiky read path, and a hot row (a viral link) becomes a repeatedly re-read single point of contention. It'll work at low volume in a demo; it will not survive a single link going viral.

---

4. Key generation — how do we mint the short code?

> **Interviewer:** Before we scale reads — how do you actually generate the code? What stops two writers from generating the same one?

Candidate: This is a small but classic distributed-systems question. Three real options:

- **A) Hash the long URL** — `code = base62(md5(longUrl + salt))[:7]`. Deterministic (same URL → same code, free dedup), but **real collision risk** at billions of rows (birthday paradox on a truncated hash) needing check-and-retry, and it lets someone predict codes for known URLs.
- **B) A global counter, base62-encoded** — `code = base62(next_id)`. Zero collisions ever, IDs unique by construction — but a single shared counter is a hot, serialized resource, and the codes are sequential-ish (guessable) unless we scramble the bits first.
- **C) Pre-allocated ID ranges (my pick)** ✅ — same idea as Snowflake / Twitter's "ticket server," without the clock complexity.

```
Each app server asks a lightweight "range allocator" for a BLOCK of IDs
(say 10,000) at startup / when it runs low:

   Server A: owns [10,000,000 – 10,009,999]     Server B: owns [10,010,000 – 10,019,999]

It then hands out IDs from its OWN range in memory — no coordination per
request. base62-encode the ID -> short code. Refill happens once per 10,000
requests, not once per request.


  App server │  Range Allocator: ONE   │  itself just does INCR-by-10000
  asks once  │  atomic counter (Redis  │  on a single counter — cheap,
  per 10k IDs│  INCRBY or a DB row)    │  called ~4/sec, not 40k/sec
  ──────────►└─────────────────────────┘
```

- **No collisions** (ranges are disjoint), **no per-request contention** — the contended resource is hit ~4 times/sec (once per 10,000 writes) instead of 40,000 times/sec, so even a single locked DB row is trivially fast at that rate. Scales linearly with app servers.
- To avoid guessable sequential codes, **XOR/shuffle the bits** of the ID before encoding — a fixed reversible permutation, still unique, no longer obviously sequential.
- I still add a **conditional write** (`IF NOT EXISTS`) as a defensive check, belt-and-suspenders since the allocator already guarantees uniqueness.

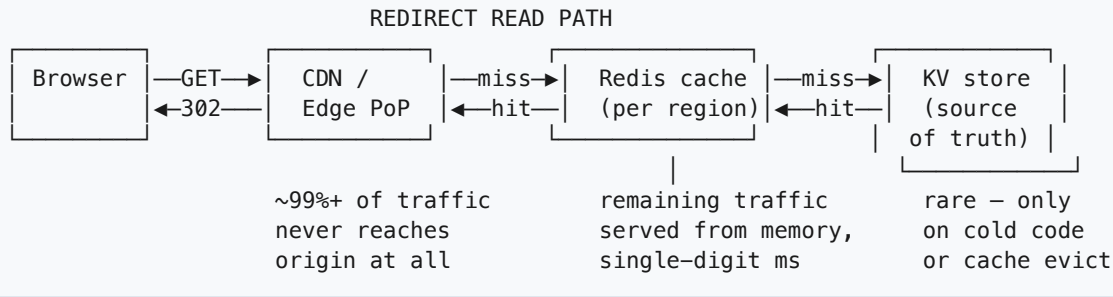
Custom aliases skip generation entirely — just a conditional write with the user's requested string as the key; if taken, 409 and ask for another.

---

5. Scaling the read path — this is the actual problem

> **Interviewer:** OK, writes are solved. Now make the redirect path survive 40,000/sec, with a handful of links taking a wildly disproportionate share.

Candidate: Redirects have a property that makes this easy to solve well: **the mapping is immutable once created.** `aB3xK9z → https://example.com/...` never changes (unless the link is deleted/expired). Immutable data is the best possible case for caching — there is no invalidation problem on the hot path, only "does the cache have it yet."



Cache-aside, read-through:

```
GET /{code}
  1. Edge/CDN: is this code cached at the edge? (only viable for 301s, see below)
  2. Redis: GET code -> longUrl
       HIT  -> respond immediately (this should be ~99%+ of traffic — power-law
               click distribution means a small set of codes dominates volume)
       MISS -> read KV store -> SET into Redis with a long TTL (mapping is
               immutable, so TTL is really just a memory-management knob, not
               a correctness one) -> respond
The cache almost never needs invalidation, because mappings don't change.
The only "invalidation" events are delete/expire, which are rare writes.
```

301 vs 302 — the tradeoff to say out loud. A **301 (permanent)** redirect gets cached by the *user's browser and by intermediate proxies/CDNs* — the second time that user clicks the same link, their browser doesn't even ask us, which is fantastic for our load but means **we lose visibility into that click** for analytics, and if we ever wanted to repoint a code we couldn't (browsers remember it forever). A **302 (temporary)** always comes back to us, so we see every click (needed for the analytics requirement) at the cost of a little more traffic. **Given we were explicitly asked for click analytics, I'd default to 302** — and that's a legitimate, common real-world choice (bit.ly does this) despite "301 is more cache-friendly" being the textbook answer.

---

6. Database choice — why a KV store, not relational

> **Interviewer:** What's actually storing the mapping — and why?

Candidate: A distributed KV store — DynamoDB or Cassandra — partitioned by `short_code`.

- The **entire access pattern** is a single-key point lookup: `short_code → long_url`. No joins, no multi-row transactions, no range scans on the hot path — the textbook KV case. Both Dynamo and Cassandra partition by key automatically, so as we add billions of rows and thousands of QPS, we add nodes, not vertical capacity to one server.
- **Why not a relational DB (Postgres/MySQL) as the primary store at this scale?** At 6B+ rows a single relational primary either needs heavy manual sharding (reinventing what Dynamo/Cassandra already do) or a beefy single writer that becomes the ceiling — and we get **none of the relational benefits**: nothing to `JOIN`, no need for cross-row ACID on "create one mapping." Paying for ACID/joins we never use while fighting sharding by hand is the wrong trade.
- If we later want **rich analytics queries** ("top 10 links this week," "clicks by referrer") — that's aggregation, not point lookup, and belongs in a **separate OLAP store** fed asynchronously, not the

hot-path KV store.

---

7. Data model

```
Table: urls   (KV / wide-column, partition key = short_code)
-------------------------------------------------------------
 short_code    STRING   PK        e.g. "aB3xK9z"
 long_url      STRING             destination
 created_at    TIMESTAMP
 expires_at    TIMESTAMP, null    TTL cleanup (native TTL feature in Dynamo/Cassandra)
 owner_id      STRING, null       for custom-alias ownership / dashboards
 is_custom     BOOL               user-chosen vs. generated
-------------------------------------------------------------
 GET by short_code -> O(1) single-partition read.

Table: click_counts   (separate store, NOT the hot-path KV table)
 short_code  STRING PK
 window      STRING     e.g. "2026-07-19"  (daily bucket)
 count       COUNTER    incremented asynchronously, never on the redirect path
```

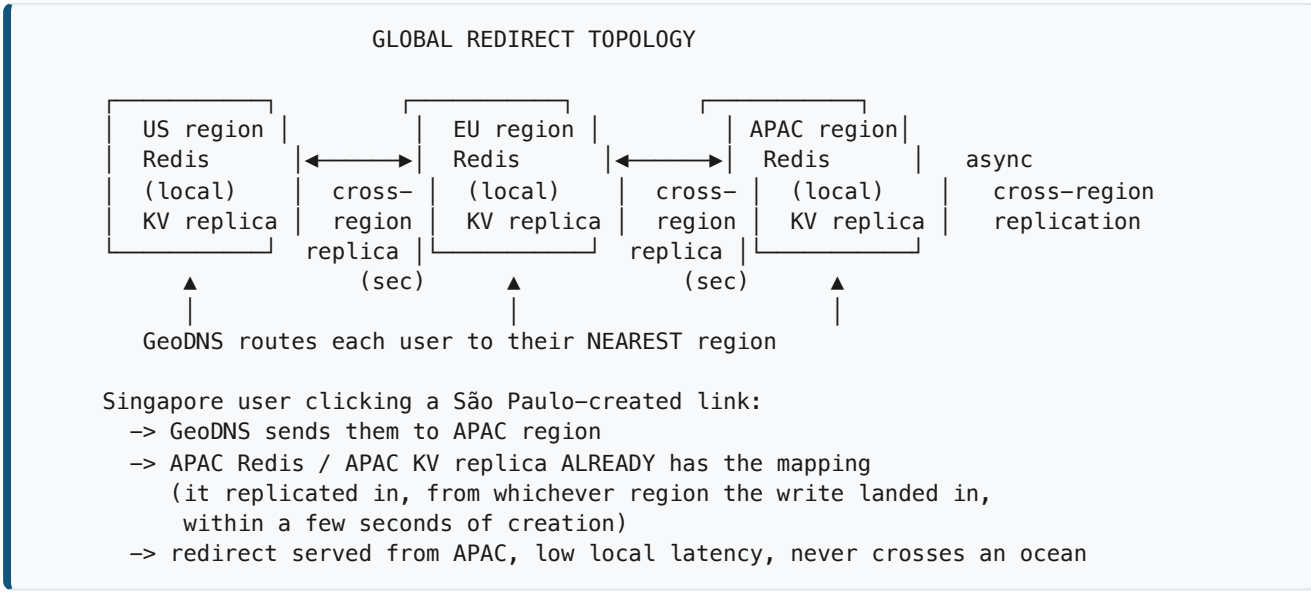
Keeping `click_counts` out of the `urls` table is deliberate — a synchronous counter increment on every redirect would turn our immutable, trivially-cacheable read into a write, defeating the entire caching story.

---

8. Multi-region — the signature concern here

> **Interviewer:** This product is used everywhere in the world. A user in Singapore clicks a link created by someone in São Paulo. Walk me through making that fast.

Candidate: This is where I lean hardest, because global redirect latency *is* the product. A shortener that's fast in us-east and slow everywhere else has failed at its one job.



- Because the mapping is **immutable and not access-controlled per-region**, I replicate the **entire dataset to every region**, active-active for reads — no reason to keep a Singapore user's redirect waiting on a round-trip to Brazil.
- **Writes go to any region** (conflict-free since codes are globally unique by construction from the range allocator) and **replicate out asynchronously** — Dynamo Global Tables or Cassandra multi-DC replication do this natively. A few seconds of propagation lag is fine: nobody expects a link to be clickable worldwide the same millisecond it's created.
- **Redis is per-region too**, warmed independently by local traffic, so even cache misses fall through to a **local** KV replica, never a cross-region hop. **CDN edge PoPs** sit in front of all of this, serving cache-friendly redirects from wherever the clicker is, with no round trip to any regional origin at all. **GeoDNS/anycast** makes the first hop short; the local-replica strategy keeps every hop after it short too.

The net effect: no matter where a link was created or clicked from, the redirect is served from infrastructure physically close to the *clicker* — one slow region is irrelevant to everyone else.

---

9. Latency — where the milliseconds go

Candidate: For a redirect, budget roughly:

```
GeoDNS / anycast routing to nearest PoP        ~0–5ms   (network-level, not app)
TLS handshake (session resumption helps)        ~0–1ms  (warm connections)
Edge/CDN cache check                            <1ms    (hit -> done here)
Redis lookup (local region)                     ~1ms
[only on cache miss] local KV replica read       ~5–10ms
------------------------------------------------------
Cache hit (the ~99% case):     ~2–7ms total     Cache miss (cold code): ~10–20ms
Both comfortably inside our <50ms budget.
```

The entire latency strategy: **make the ~1% miss path acceptable, and make the 99% hit path served entirely from memory, close to the user** — cache aggressively, replicate globally, keep the miss path shallow (one KV hop, not a chain of services).

---

10. Consistency — what's strong, what's eventual

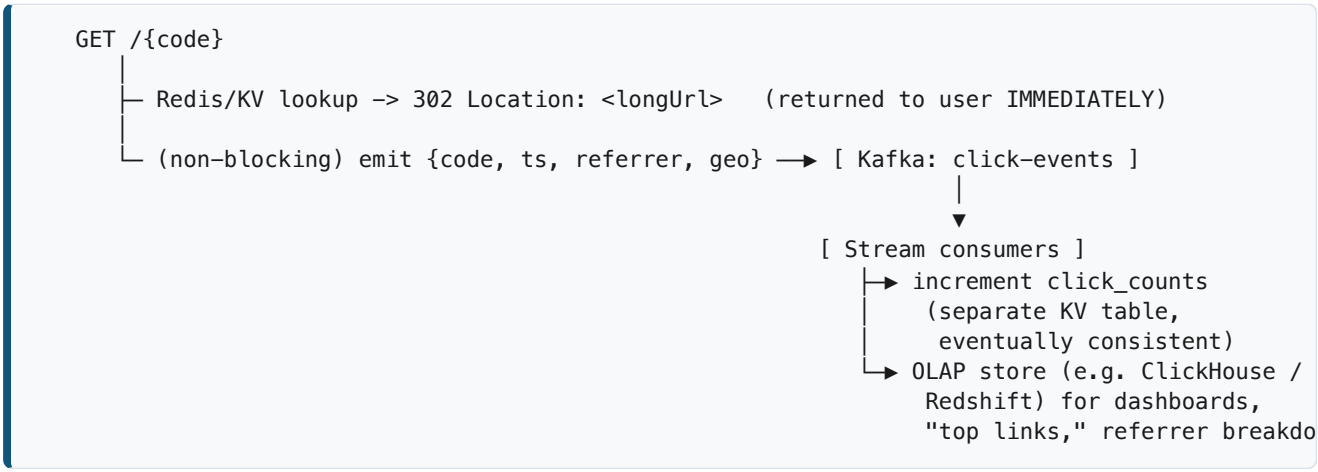
- **Strong, needs-to-be:** code uniqueness at creation time (the range allocator + conditional write give us this without a global lock).
- **Eventual, and totally fine:** cross-region replication of the mapping. A brand-new link not resolving everywhere for a few seconds is invisible to users — nobody clicks a link before it's shared. Immutability makes this safe: a stale replica still returns the *correct* answer, just possibly a few seconds late to exist (never a *wrong* answer).
- **Eventual, by design:** click counts — explicitly allowed to be approximate and delayed, which is what lets us keep counting entirely off the hot path.

---

11. Async — click analytics off the critical path

> **Interviewer:** Every redirect should count as a click. How do you track that without slowing down the redirect?

Candidate: The redirect response goes out **first**; the click event is fired-and-forgotten into a stream, and aggregation happens entirely off to the side.



- **Why Kafka:** high-throughput, ordered-per-key, durable, and replayable — if a consumer falls behind or crashes, events aren't lost and we can reprocess. Partition by `short_code` so counts for a given link stay ordered.
- **Why not increment a counter synchronously in the KV store on every redirect?** Because that turns our immutable, cacheable read into a write on the hottest path in the system — exactly what we spent sections 5-9 avoiding. Even a "fast" atomic increment, done 40,000 times/sec against a handful of hot keys, reintroduces the hot-key contention problem this design otherwise sidesteps.
- Click counts shown to a link's owner ("142,003 clicks") are therefore **eventually consistent by a few seconds to a minute** — acceptable, exactly as scoped.

---

12. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Mapping store (source of truth)	**KV store — DynamoDB or Cassandra**, partitioned by `short_code`	Access pattern is a pure single-key lookup at massive read QPS; both partition automatically for horizontal scale. *Not relational (Postgres/MySQL)* at billions of rows — no joins/ACID needed here, and a relational primary becomes a manual-sharding project for no benefit; we'd pay ACID/join overhead we never use.

Concern	Choice	Why this, and why not the alternative
Hot redirect cache	**Redis, per-region, cache-aside, long TTL**	Mapping is immutable, so caching has no invalidation problem — pure upside. Memory-speed lookups (~1ms) absorb ~99%+ of the 40k/s peak before the KV store ever sees it. *Not skipping the cache* — every redirect would hit the KV store directly, and a viral link becomes a hot-partition problem.
Global edge	**CDN / anycast edge PoPs**	Serves cache-friendly redirects (and static assets, preview pages) from the location nearest the *clicker*, shaving the cross-region round trip entirely. *Not origin-only serving* — every click would pay full backbone latency regardless of how close the user is.
Key generation	**Pre-allocated ID ranges (ticket-server style) + base62 encode**, defensive conditional write	Turns a per-request contended counter into a per-10,000-requests one; zero collisions by construction; scales linearly with app servers. *Not a naive single global counter* — every write serializes on it. *Not hash-of-URL alone* — real collision risk at billions of rows, needs retry logic.
Custom alias uniqueness	**Conditional write** (`IF NOT EXISTS`) on the KV store	O(1), no coordination service needed since the "key" is user-chosen, not generated. *Not check-then-insert in app code* — race between the check and the insert; the atomic conditional write closes that window.
Click analytics pipeline	**Kafka → stream consumers → OLAP store (e.g. ClickHouse/Redshift) + async counter table**	Keeps every write off the redirect's critical path; durable + replayable; partition-by-code keeps per-link ordering. *Not a synchronous increment on the hot KV table* — reintroduces exactly the hot-key contention problem caching was meant to eliminate.
Replication topology	**Multi-region active-active reads** (Dynamo Global Tables / Cassandra multi-DC), async cross-region replication	Immutable data + eventual consistency across regions is safe and invisible to users; every region serves reads locally, at local latency. *Not a single-region primary with cross-region reads* — every redirect from a far region pays a cross-ocean round trip, blowing the latency budget.
Load balancing / edge	**L7 / API Gateway**, `POST /shorten` and `GET /{code}` routed to separate autoscaling pools	Path-based routing lets the read pool (huge, latency-sensitive) scale independently from the tiny write pool; gateway also does TLS termination, rate limiting the create API, and auth for owned links. *Not L4 alone* — can't route by path or terminate TLS; L7 is where the actual routing decisions live.

Say-it-out-loud justification: "*This is a read-scaling problem wearing a URL-shortener costume — 100:1 reads to writes, so I cache the immutable mapping aggressively in per-region Redis behind a global CDN, backed by a horizontally-partitioned KV store for the rare cache miss. Uniqueness comes from pre-allocated ID ranges so no single counter is ever contended. Multi-region active-active reads with async replication mean every user gets served from infrastructure physically close to them, and eventual consistency is free because the data never changes once written. Click analytics are pushed entirely off the hot path through Kafka so the redirect itself never does more than one cache-speed lookup.*"

13. Failure modes & graceful degradation

Failure	What happens	Mitigation
Redis (regional) down	Cache misses spike	Falls back to local KV replica reads — slower (~10-20ms) but still correct; Redis restarts and re-warms from traffic naturally (cache-aside self-heals)
One KV node down	Partition owned by that node degraded	Replication factor (RF=3) + quorum reads keep serving from surviving replicas
Whole region down	Users in that region would be stranded	GeoDNS reroutes to the next-nearest region; data is already replicated there, so redirects keep working, just from a slightly farther PoP
Write path (range allocator / create service) down	New link creation pauses	**Redirects are completely unaffected** — reads don't depend on the write path at all; creation requests queue or fail cleanly and retry once healthy
Kafka / analytics pipeline down or lagging	Click counts stale/delayed	User-facing redirect is unaffected (analytics is async and was already eventually consistent); consumers catch up and backfill once healthy
Code doesn't exist / expired	—	Return a clean 404 / "this link has expired" page — never a 500; this is a normal, expected outcome, not an error

Design principle, same as most read-heavy systems: **the read path must be able to survive total failure of the write path**, because a broken redirect breaks every link that already exists in the wild, right now, everywhere it was ever shared.

14. Follow-up curveballs

> **Interviewer:** A marketing link goes viral — one code getting 100k clicks/sec. Does your design survive that?

Candidate: Yes, and better than most parts of the system, because this is exactly what caching was built for: that one key sits warm in **every regional Redis** and likely at the **CDN edge** too, so 100k/sec on a *single* key is actually the *easiest* traffic pattern — it's memory-speed reads on one hot-but-cached key, spread across regions and edge PoPs. It would only hurt us if we were reading the KV store directly per request, which is exactly why the cache sits in front.

> **Interviewer:** How would you delete billions of expired links without a giant slow batch job?

Candidate: Use the KV store's **native TTL feature** (Dynamo TTL / Cassandra TTL) on the `expires_at` attribute — the store expires and reclaims rows in the background, no application-level batch scan needed. On the read path, I'd also actively check `expires_at` so an item isn't served stale in the (short) window before physical TTL cleanup runs.

> **Interviewer:** How do you stop someone from shortening a malicious/phishing URL?

Candidate: Check the long URL against a safe-browsing / threat-intel API **at creation time** (async is fine — flag and can retroactively disable if it comes back malicious a moment later, since we don't want link creation to block on a third-party API). Rate-limit the create endpoint per user/IP at the gateway to stop mass-abuse. This is a write-path concern and doesn't touch the read-path design at all.

> **Interviewer:** Could two different regions generate the same short code at the same time?

Candidate: No, by construction — each app server (in any region) is handed a disjoint ID range by the allocator, and the allocator's counter is the single global source of "next range," so ranges never overlap regardless of which region requested them. Combined with the defensive conditional write, this is safe even under a network partition between regions, worst case just a slower allocator round-trip for whichever region is momentarily cut off.

---

15. Deep-dive: "Why not just hash the long URL for the short code?"

> **Interviewer:** Section 4 mentioned hashing as an option and moved past it quickly. Let's stay there. Why not just do `code = base62(md5(longUrl))[:7]` and call it done? No allocator, no range bookkeeping, no moving parts — just a pure function of the input.

Candidate: It's the first thing everyone proposes, because it *feels* stateless and elegant. It falls apart on three separate axes once you push the numbers to our scale, and it's worth walking through all three, because they fail for different reasons.

Axis 1 — collision math actually bites at billions of rows

Truncating a hash to 7 base62 characters gives us a space of `62^7 ≈ 3.5 trillion` values — looks huge, but the birthday paradox says collision probability grows with the *square* of the item count, not linearly:

```
Birthday-paradox approximation:  P(collision) ≈ n² / (2 * N)
  N = 3.5 * 10^12   (7-char base62 space)
  n = 6 * 10^9       (our 5-year estimate from section 2)

  P(collision) ≈ (6e9)² / (2 * 3.5e12)
               ≈ 3.6e19 / 7e12
               ≈ 5,140          <- expected number of COLLIDING PAIRS,
                                       not a probability near zero!

At 6B rows, we don't get "maybe one unlucky collision" — we get thousands
of them, guaranteed, as a simple consequence of the pigeonhole math.
```

That means hashing **cannot** be collision-free at our volume — collisions aren't a rare edge case to shrug off, they're an expected, recurring event that the write path must handle every single day.

Axis 2 — the collision check is read amplification on every write

Because collisions are expected, every write must **read before it writes**: hash the URL, then `GET` that code to see if it's already taken by a *different* long URL, and if so, retry with a different salt/suffix and check again.

```
Naive hash write:                        Hash-with-collision-check write:
  hash(longUrl) -> code                    hash(longUrl) -> code
  INSERT(code, longUrl)                    GET code  ─┐
  (1 write, correctness NOT guaranteed)       exists, same URL? -> reuse, done
                                               exists, DIFFERENT URL? -> re-salt, GET again
                                               (retry loop, unbounded worst case)
                                             INSERT(code, longUrl)
                                            (1 read + N reads per collision + 1 write)
```

Our write volume (~40/s) makes this cheap in isolation — but it's *strictly worse* than the allocator, which does **zero** reads on the request path (section 4), and the retry loop has no hard upper bound: under adversarial or just unlucky input, a single request could spin through several collisions before landing a free code.

Axis 3 — determinism is a feature we don't actually want

Hashing gives `same longUrl -> same code`, which sounds like free dedup, but it actively conflicts with two requirements we already have:

- **Two campaigns, same destination, different codes.** Marketing routinely wants `bit.ly/spring-sale` and `bit.ly/newsletter-promo` to both point at the same landing page, with **separate click analytics per code**. A deterministic hash gives them the *same* code for both — we'd have to bolt on a salt-per-request anyway, at which point we've thrown away the one benefit (determinism) and kept all the costs (collision checks).
- **Custom aliases live in the same namespace.** A user-chosen alias like `bit.ly/my-brand` is an arbitrary string with no relationship to any hash. The generator has to coexist with fully custom keys either way, so "generation is a pure function of the input" buys us nothing structurally — we still need a conditional write and a shared keyspace check.

Comparison table

Concern	Hash-based (`md5(url)` truncated)	Pre-allocated counter ranges (our pick)
Collision behavior at 6B rows	Expected, recurring (birthday paradox) — thousands of colliding pairs, not zero	None by construction — each ID is issued exactly once, ranges are disjoint
Request-path reads	Read-before-write required to detect collisions; retry loop is unbounded worst case	Zero reads per request — IDs come from an in-memory range already owned by the server
Contended resource	The collision-check itself becomes contention if the same URL is submitted concurrently	Allocator hit once per 10,000 writes (~4/s), never touched per-request
Predictability / guessability	Same input -> same output; an attacker can precompute codes for known URLs	Sequential-looking unless bit-shuffled; shuffle step defeats guessing either way

Concern	Hash-based (`md5(url)` truncated)	Pre-allocated counter ranges (our pick)
Supports "two codes, one URL"	No — needs an artificial salt bolted on, undoing the "pure function" appeal	Yes, trivially — every request just gets the next ID regardless of URL content
Coexistence with custom aliases	No structural advantage — custom keys still need their own conditional-write path	Same — but generation was never pretending to unify with custom keys anyway
Operational moving parts	"Stateless," but hides state in the collision-retry logic	One small allocator service/counter; simple, well-understood (Snowflake-style)

Why not just a single DB auto-increment `SEQUENCE`?

This is the middle-ground option worth naming explicitly: skip both hashing *and* the allocator, and let Postgres/MySQL hand out IDs via a native `AUTO_INCREMENT` / `SEQUENCE`. It's collision-free, same as the allocator — but it reintroduces the exact failure mode from section 3's naive V1: **one sequence is one row/counter in one place**, and every single writer, across every app server and every region, serializes through it to get the next value. At 40 writes/sec globally that single point *happens* to be fine today — but it's a **global synchronous dependency for every write, in every region**, with no natural way to shard it (values must stay monotonically assigned from one source), and it becomes the one thing standing between "add another app server" and "actually higher write throughput." The allocator gets the same collision-free guarantee while making the contended operation ~2,500x rarer (once per range, not once per row) and letting each app server mint IDs entirely in memory in between.

> **Rule of thumb:** *Hashing looks free because it's stateless, but "stateless" just means the state moved into a collision-retry loop on the write path. A pre-allocated range turns a per-request contended resource into a per-10,000-requests one — that's the only number that matters at scale, and it beats both hashing and a naive DB sequence on every axis except "zero moving parts," which stops being true the moment you add the collision handling hashing actually requires.*

---

16. Deep-dive: the viral cold link (thundering herd) and the malicious lookup (cache penetration)

> **Interviewer:** Section 9 assumed a cache miss costs ~10-20ms and moves on. Walk me through the worst version of a miss: a **brand-new link that goes viral in the first second**, before it's ever been cached anywhere — and separately, what stops someone from just requesting **codes that don't exist** over and over?

Candidate: These are two different failure shapes that both show up as "the cache isn't helping," so let me take them one at a time.

Case A — the hot-new-key thundering herd

A link is created, cache-aside means we only populate Redis **on first read**, and the code is immediately shared to a huge audience (a celebrity tweets it) before that first read has happened anywhere.

```
t=0ms     link created, written to KV store. Redis: NOT cached yet (cache-aside
          only populates on read, and nobody has read it yet).

t=50ms    the tweet goes out. 100,000 clients, essentially simultaneously, issue
          GET /aB3xK9z

                    100,000 concurrent requests
                             │
                             ▼
                  ┌──────────────────────┐
                  │   Redis: MISS        │  <- same key, same instant, every request
                  │   (all 100k asked    │     independently decides "not cached,
                  │    at once)          │     go to KV store"
                  └──────────────────────┘
                             │
                  100,000 SIMULTANEOUS reads
                             ▼
                  ┌──────────────────────┐
                  │   KV store           │  one partition (this one key) absorbs
                  │   ONE hot partition  │  all 100k reads in the same instant —
                  └──────────────────────┘  exactly the hot-partition problem
                                              caching was supposed to prevent
```

The cache isn't wrong here — it's just **too slow to fill**. Every one of those 100k requests independently sees "miss" and independently goes to the KV store, because none of them knows another request is *already* fetching the same key. We paid for a cache and got a hot-partition stampede anyway, on the one key we most needed protected.

Case B — cache penetration (looking up keys that don't exist)

A completely different shape: someone (bot, scanner, or just a scraper hitting expired links) requests codes that **never existed or were deleted**.

```
GET /zzzzz99   (never existed)
     │
     ▼
  Redis: MISS  (there is nothing to cache — a miss on a real key eventually
                becomes a hit once populated; a miss on a FAKE key is a miss
                every single time, forever)
     │
     ▼
  KV store: MISS too, returns "not found"
     │
     ▼
  (nothing gets cached, because cache-aside only caches SUCCESSFUL lookups)

Repeat 50,000 times/sec with random/incrementing fake codes:
  -> every single request bypasses the cache entirely and lands on the KV
     store, indistinguishable at the cache layer from real traffic
```

This is quieter than Case A but more dangerous in a way: it's a **standing hole that never self-heals**, because a cache-aside strategy has no mechanism for caching absence. An attacker doesn't need a

viral link — they just need to know the cache only helps when the answer is "yes."

Ranked fixes

1. Cache-on-write for new links (kills Case A at the root). Don't wait for the first reader to discover the key is missing — when `POST /shorten` succeeds, **immediately write the mapping into every regional Redis** (or at least the region the write landed in, replicating out), not just the KV store. A link is never "cold" in cache after the millisecond it's created, so the viral-at-t=0 scenario can't happen — there is no window where the key exists but isn't cached.

2. Request coalescing / single-flight (the safety net for whatever slips through). For any given key, only let **one** in-flight request per cache-server actually go to the KV store; every concurrent request for the *same* key attaches to that one in-flight fetch and gets the same result when it resolves.

```
100,000 requests for aB3xK9z arrive within the same ~10ms window:

  request #1   -> "no fetch in flight for aB3xK9z" -> issues the KV read,
                  registers itself as the in-flight owner
  requests #2..100,000
               -> "fetch already in flight for aB3xK9z" -> just WAIT on #1's
                  result -> all 100,000 respond from ONE KV read, not 100,000
```

Libraries call this "single-flight" (Go's `singleflight`, or building it on a Redis lock / `SET NX` per key while the fetch is in progress). This is the general-purpose fix — it helps even if cache-on-write somehow missed a key, and it costs nothing on the normal cache-hit path.

3. Negative caching for missing keys (kills Case B). When the KV store says "not found," cache **that fact** too, with a short TTL (seconds, not the long TTL used for real mappings): `SET missing:{code} 1 EX 30`. The next 50,000 requests for the same fake code in that window hit Redis and bounce off a cached "no," never touching the KV store at all. Short TTL bounds how long a *newly created* code could theoretically be mistaken for missing (only relevant if cache-on-write, fix #1, somehow didn't run — belt-and-suspenders).

4. Bloom filter in front of the cache (cheapest per-lookup, best against scanning abuse). A Bloom filter over the set of *all valid codes* answers "definitely not present" with a single in-memory bit check, no network hop at all — a scanner hammering random/sequential fake codes gets rejected before it even reaches Redis. False positives are possible (rare "maybe exists," falls through to the normal path) but **false negatives are impossible** — a real code never gets wrongly rejected. Needs a rebuild/insert on every new code, which is cheap at our ~40 writes/sec.

Recommendation: cache-on-write (#1) should mean Case A basically never happens by construction; single-flight (#2) is cheap insurance I'd add regardless, since it protects against any key going hot faster than it can be populated (not just brand-new ones — an existing link can also go viral after an eviction). Negative caching (#3) is the simplest fix for Case B and I'd ship it first; a Bloom filter (#4) is the next layer if we're seeing sustained scanning/abuse traffic that even negative caching's cache layer shouldn't have to absorb.

> **What to say out loud:** "*These are two different failure modes wearing the same 'cache miss' costume. The viral-cold-link case is a timing problem — the cache didn't have time to warm — so I fix it by never letting there be a window where a link exists but isn't cached: write to cache synchronously at creation time, and add request coalescing as a general backstop for any key that*

> *goes hot faster than it can be populated. The nonexistent-key case is a fundamentally different problem — cache-aside has no notion of caching absence — so it needs its own fix: negative caching for the common case, and a Bloom filter if someone's actively scanning for valid codes.*"

Summary — the mental model

A URL shortener is **not a hard write problem — it's a "make an immutable single-key lookup survive a 100:1 read skew, globally, in single-digit milliseconds" problem.** The winning moves: generate collision-free codes via **pre-allocated ID ranges** (no single contended counter), store the mapping in a **horizontally-partitioned KV store** sized for point lookups, and **cache aggressively** — per-region Redis plus a global CDN — because the data never changes once written, so caching has zero invalidation cost. **Multi-region active-active replication** puts every user's redirect close to them; **eventual consistency** across regions is free because stale just means "not yet replicated," never "wrong." Click analytics move **entirely off the hot path** through Kafka, so the one thing every stranger on the internet is waiting on — the redirect itself — never does more than a cache-speed lookup.